



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

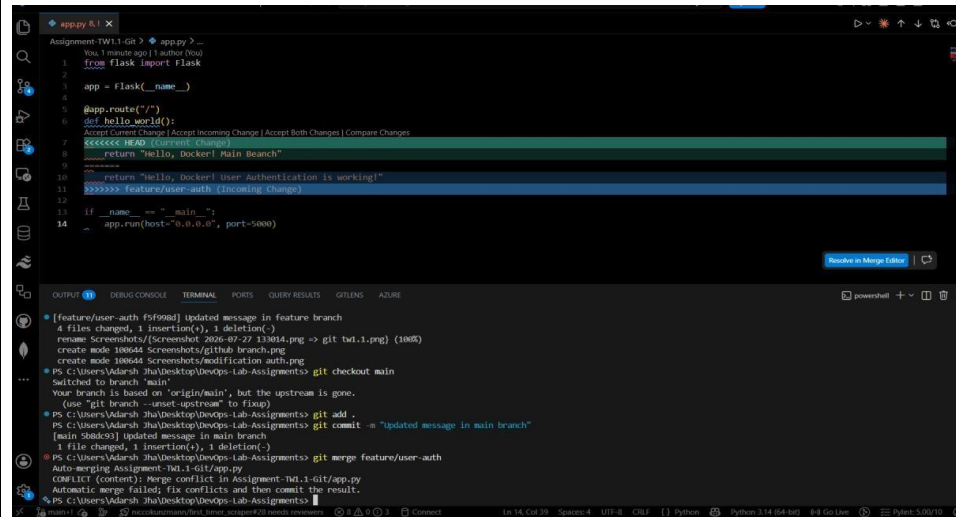
Assignment No. 02

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Adarsh Jha
PRN No.	23070122261
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	16 th July, 2026
Title of Assignment:	Git Branching & Merging
Theory:	Git branching is a feature that allows developers to work on new features or bug fixes independently without affecting the main project. A branch creates a separate line of development where changes can be made safely. Once the work is completed and tested, the branch can be merged back into the main branch using the git merge command. This workflow helps in parallel development, better collaboration, and maintaining a stable main branch. Commands Used 1. git status – Checked the current repository status and confirmed that the working tree was clean and the local main branch was up to date. 2. git checkout -b feature-update – Created a new branch named feature-update and switched to it. 3. git branch – Displayed all local branches and verified that feature-update was the active branch. 4. git status – Checked the repository status after modifying the README.md file and confirmed that the changes were detected. 5. git add README.md – Added the modified README.md file to the staging area. 6. git commit -m "Update README on feature branch" – Committed the

	staged changes to the feature-update branch with a descriptive commit message. 7. git status – Verified that the working tree was clean after committing the changes. 8. git checkout main – Switched back to the main branch. 9. type README.md (<i>Windows</i>) – Displayed the contents of the README.md file to verify that the changes from the feature branch were not yet present. 10. git merge feature-update – Merged the feature-update branch into the main branch using a fast-forward merge. 11. type README.md (<i>Windows</i>) – Verified that the merged changes were now present in the README.md file. 12. git push origin main – Pushed the updated main branch with the merged changes to the remote GitHub repository.
Implementation:	Step 1: Verify Current Repository Status Verified Repository Status — Checked the repository status and confirmed that the working tree was clean and the local main branch was up to date with the remote repository. Step 2: Create and Switch to a New Branch Created Feature Branch — Created a new branch named feature-update and switched to it successfully using git checkout -b. Step 3: Modify README.md Modified README.md — Updated the README.md file on the feature-update branch and verified that Git detected the modified file. Step 4: Stage and Commit the Changes Committed Feature Changes — Staged and committed the updated README.md file on the feature-update branch with a descriptive commit message. Step 5: Switch Back to Main Branch Switched to Main Branch — Switched back to the main branch and verified that the feature branch changes were not yet available. Step 6: Merge the Feature BranchMerged Feature Branch — Successfully merged the feature-update branch into the main branch using a fast-forward merge. Step 7: Push Updated Main BranchPushed Updated Main Branch — Uploaded the merged changes from the local main branch to the remote GitHub repository.

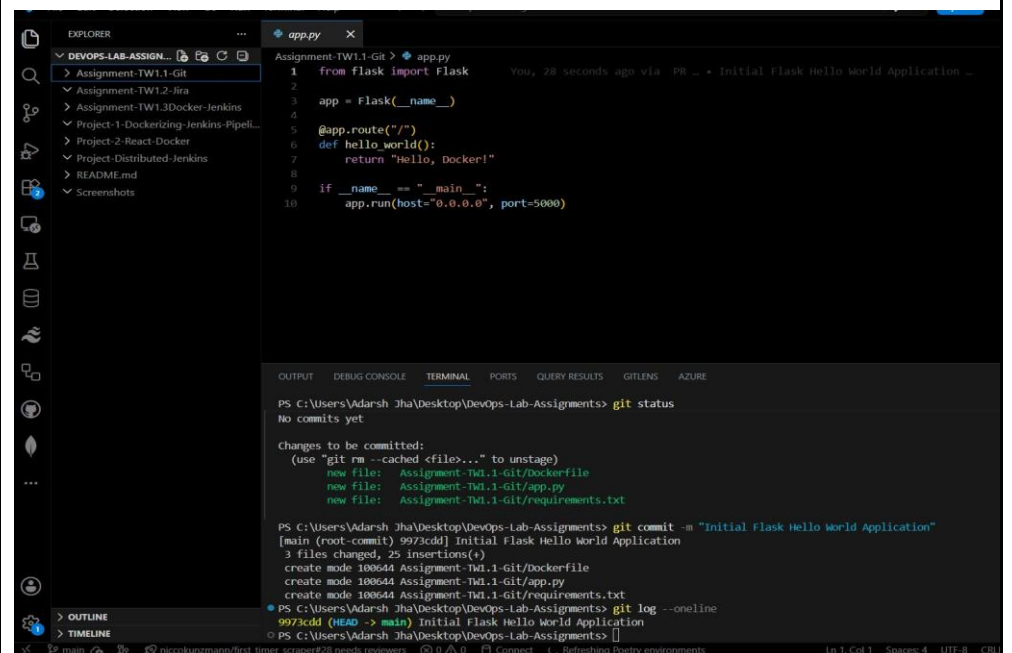
Step 8: Verify on GitHub

Repository Verified — Verified on GitHub that the merged changes are present in the main branch and that the commit history reflects both commits.



```
app.py 8:1 X
Assignment-TW1.1-Git > app.py >
You, 1 minute ago | 1 author (You)
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def hello_world():
7     Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
8     <----- HEAD (feature/user-auth)
9     return "Hello, Docker! Main Branch"
10
11     <----- HEAD (feature/user-auth)
12     return "Hello, Docker! User Authentication is working!"
13
14 if __name__ == "__main__":
15     app.run(host="0.0.0.0", port=5000)

[feature/user-auth fsf998d] updated message in feature branch
4 files changed, 1 insertion(+), 1 deletion(-)
rename Screenshots/Screenshot 2026-07-27 133014.png -> git twi.1.png (100%)
create mode 100644 Screenshots/gitHub branch.png
create mode 100644 Screenshots/modification auth.png
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git checkout main
Switched to branch 'main'
Your branch is based on 'origin/main', but the upstream is gone.
(use "git branch --unset-upstream" to fixup)
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git add .
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git commit -m "updated message in main branch"
[main 5b8dc93] updated message in main branch
1 file changed, 1 insertion(+), 1 deletion(-)
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git merge feature/user-auth
Auto-merging Assignment-TW1.1-Git/app.py
CONFLICT (content): Merge conflict in Assignment-TW1.1-Git/app.py
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments>
```



```
EXPLORER
DEVOPS-LAB-ASSIGN...
> Assignment-TW1.1-Git
  Assignment-TW1.2-Jira
  Assignment-TW1.3-Docker-Jenkins
  Project-1-Dockerizing-Jenkins-Pipeli...
  Project-2-React-Docker
  Project-Distributed-Jenkins
  README.md
  Screenshots

app.py
Assignment-TW1.1-Git > app.py
You, 28 seconds ago via PW ... • Initial Flask Hello World Application ...
1 from flask import Flask
2
3 app = Flask(__name__)
4
5 @app.route("/")
6 def hello_world():
7     return "Hello, Docker!"
8
9 if __name__ == "__main__":
10     app.run(host="0.0.0.0", port=5000)

PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git status
No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Assignment-TW1.1-Git/Dockerfile
        new file:   Assignment-TW1.1-Git/app.py
        new file:   Assignment-TW1.1-Git/requirements.txt

PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git commit -m "Initial Flask Hello World Application"
[main (root-commit) 9973cdd] Initial Flask Hello World Application
3 files changed, 25 insertions(+)
create mode 100644 Assignment-TW1.1-Git/Dockerfile
create mode 100644 Assignment-TW1.1-Git/app.py
create mode 100644 Assignment-TW1.1-Git/requirements.txt
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments> git log --oneline
9973cdd (HEAD -> main) Initial Flask Hello World Application
PS C:\Users\Adarsh Jha\Desktop\DevOps-Lab-Assignments>
```